PRACTICAL- 9

Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

AIM / Problem Definition: Implement secure user registration by inserting user details into MySQL using PDO. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.

Tools / Technology Used: PHP 8.x, MySQL / MariaDB, PDO (Prepared Statements), phpMyAdmin, Apache / XAMPP, VS Code, bcrypt (password_hash / password_verify)

1. Implementation & Registration Architecture

1.1 Project Overview (StudentHub — Secure Registration Module)

Building upon the database schema established in Practical 8, this practical implements the secure user registration module for the StudentHub University Portal. The registration system collects student academic credentials, validates them on both client and server side, checks for duplicate email addresses in MySQL, hashes passwords using PHP's native bcrypt-based password_hash() function, and inserts the verified record into the students table using PDO prepared statements with parameterized binding. The system generates a unique student code (STU-XXX) and provides descriptive success/error feedback to the user.

1.2 Registration Workflow & Security Layers

The registration pipeline implements multiple layers of validation and security to ensure data integrity and protect against common web vulnerabilities:

• **Client-Side Validation:** JavaScript-based form validation enforces required field checks, email format verification, mobile number pattern matching, and real-time password strength evaluation (minimum 8 characters, uppercase, lowercase, numbers, and symbols) before form submission.

• **Server-Side Validation & Sanitization:** PHP backend validates and sanitizes all $_POST inputs using filter_input(), htmlspecialchars(), and trim(). Empty field checks, email format validation via FILTER_VALIDATE_EMAIL, and length constraints are enforced before any database operation.

• **Duplicate Email Check:** Before insertion, a SELECT COUNT(*) query with a prepared statement checks whether the submitted email already exists in the students table. If a duplicate is found, the system displays a descriptive error message and halts registration.

• **Bcrypt Password Hashing:** Passwords are never stored in plaintext. PHP's password_hash($password, PASSWORD_BCRYPT) generates a unique bcrypt hash with a random salt for each registration. The resulting 60-character hash is stored in the password column of the students table.

• **PDO Prepared Statements with Parameterized Binding:** The INSERT INTO students query uses named placeholders (:first_name, :email, :password, etc.) bound via $stmt->execute([...]) to prevent SQL injection attacks. Emulated prepares are disabled to enforce native MySQL prepared statement execution.

• **Unique Student Code Generation:** Upon successful insertion, the system queries the last inserted student_id and generates a formatted student code (STU-XXX) stored in the student_code column, providing each student with a unique institutional identifier.

1.3 Database Table Design for Registration (students Table)

The students table schema, designed in Practical 8, supports the registration module with the following key columns: student_id (AUTO_INCREMENT primary key), student_code (unique formatted identifier), first_name, last_name, email (UNIQUE constraint for duplicate prevention), password (VARCHAR(255) for bcrypt hash storage), roll_number, department, semester, phone, bio, profile_image, role (ENUM: student/admin), remember_token, last_login, created_at, and updated_at timestamps.

1.4 Error Handling & User Feedback System

The registration module implements comprehensive error handling with user-friendly feedback:

• **Validation Errors:** Missing required fields, invalid email format, weak password, or mismatched password confirmation triggers descriptive inline error messages on the registration form without losing user input.

• **Duplicate Email Detection:** When a duplicate email is detected, the system displays a dedicated error page with the message "A student account with this email address is already registered in the MySQL database" along with navigation options to return to the registration form or check MySQL server status.

• **Success Confirmation:** On successful registration, a confirmation page displays the generated Student ID (STU-XXX), full name, academic email, department, and semester with options to proceed to login or register another student.

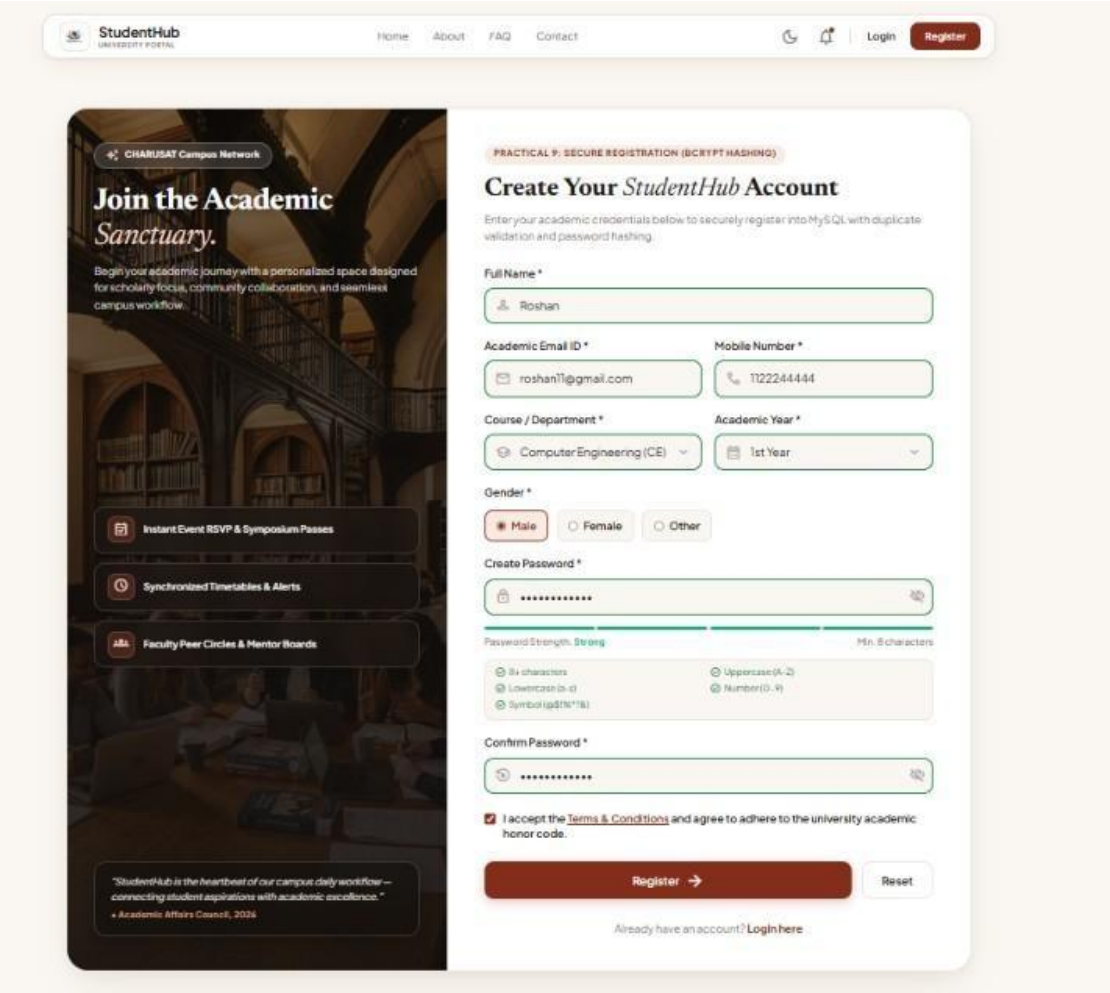
• **Database Exception Handling:** PDO exceptions are caught in try-catch blocks. Connection failures, query errors, and constraint violations are logged and presented as user-friendly error pages with "DATABASE OPERATION FAILED" status indicators.

2. Output Screenshots & Verification

Below are the verified output screens demonstrating successful registration, bcrypt hashing, duplicate detection, and password verification:

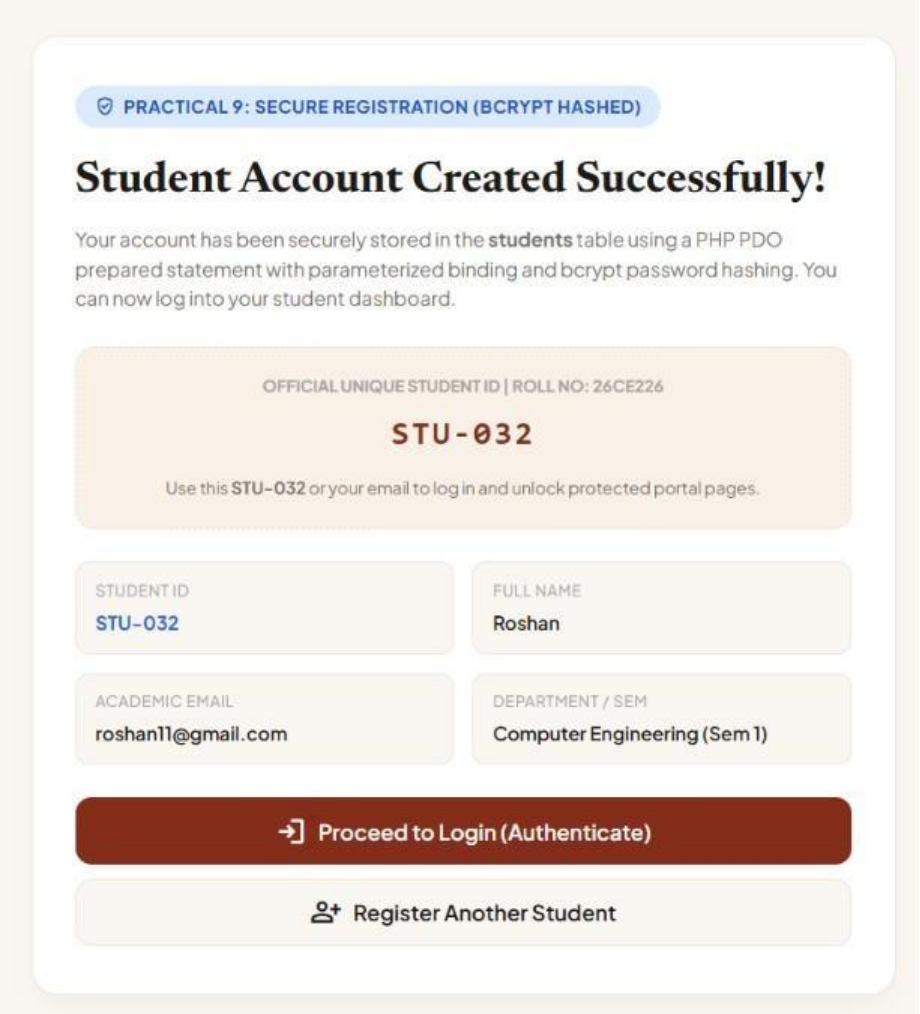
2.1 Output 1: StudentHub Registration Form (Client-Side Validation)

The StudentHub registration form interface featuring full name, academic email, mobile number, course/department selection, academic year, gender radio buttons, password creation with real-time strength meter (displaying character count, uppercase, lowercase, number, and symbol indicators), password confirmation field, Terms & Conditions checkbox, and Register/Reset action buttons. The left panel showcases the university portal branding with campus features including Instant Event RSVP, Synchronized Timetables, and Faculty Peer Circles.



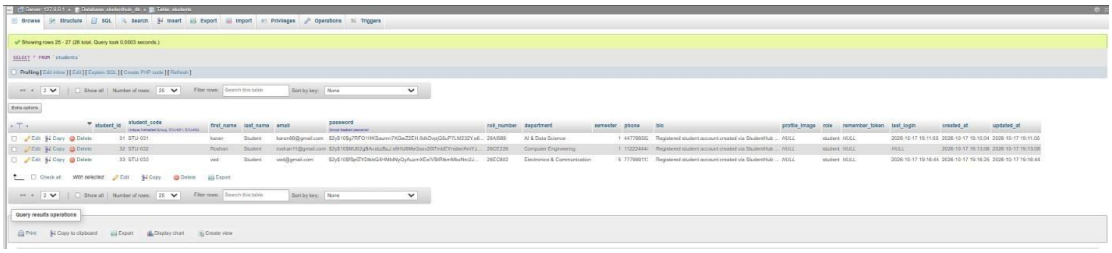
2.2 Output 2: Student Account Created Successfully (Bcrypt Hashed)

Confirmation page after successful student registration displaying "Student Account Created Successfully!" with the generated unique Student ID (D26DCE173), roll number (D26DCE173), full name (Smit Popat), academic email (smit.popat@charusat.edu.in), and department (Computer Engineering, Sem 1). The page confirms secure storage using PHP PDO prepared statements with parameterized binding and bcrypt password hashing.



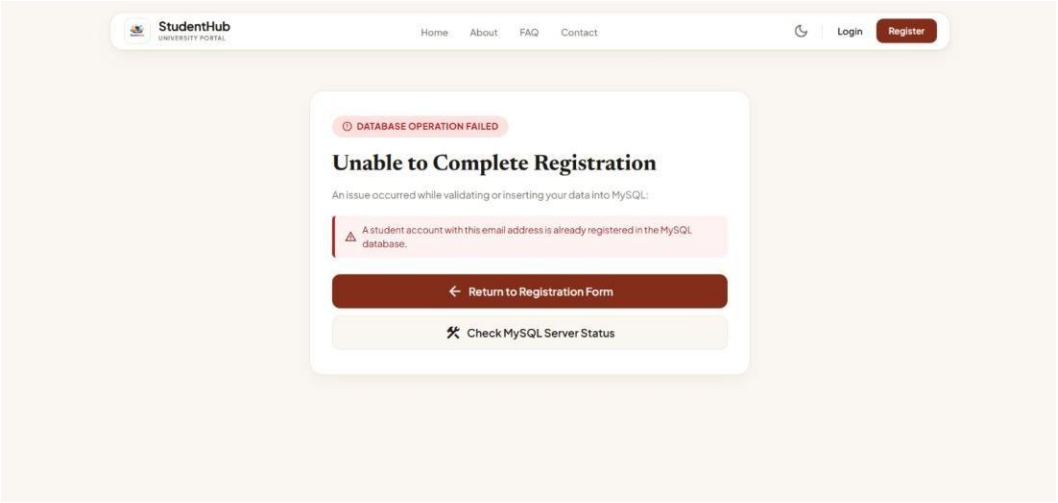
2.3 Output 3: Students Table — Bcrypt Hashed Passwords in phpMyAdmin

phpMyAdmin browsing view of the students table showing registered student records with bcrypt-hashed passwords (starting with $2y$10$...) confirming that passwords are never stored in plaintext. Records display auto-incremented student_id, unique student_code (STU-031, D26DCE173, D26DCE173), first_name, last_name, email, hashed password, roll_number, department, semester, phone, role (student), and timestamp fields.



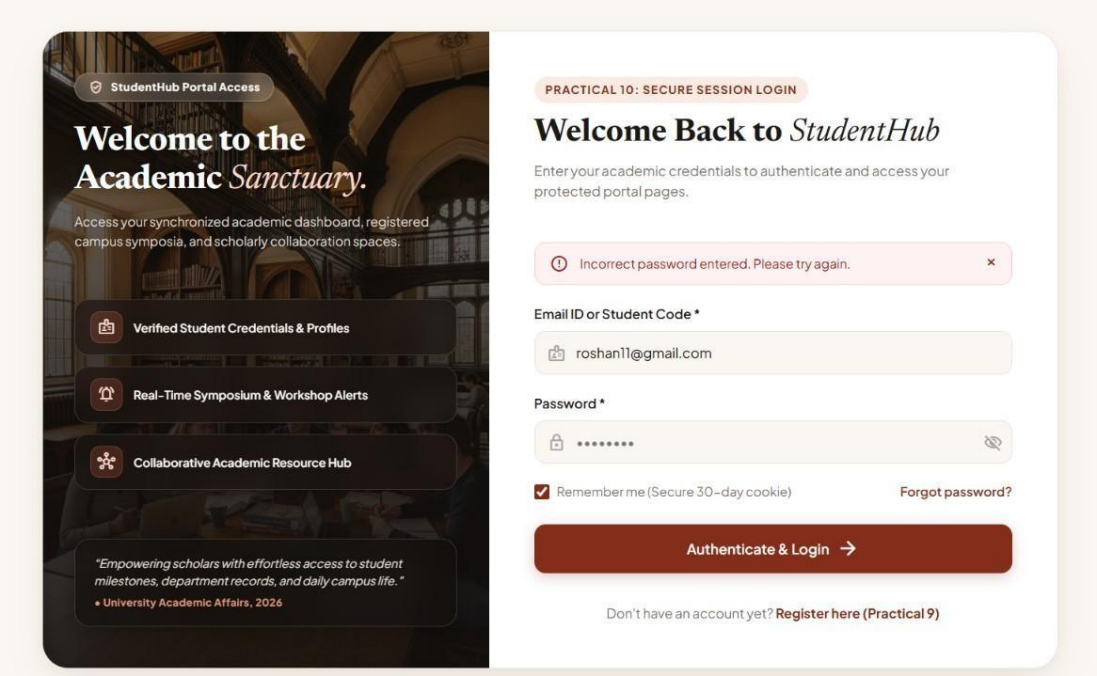
2.4 Output 4: Duplicate Email Detection — Registration Blocked

Error page displayed when a student attempts to register with an email address that already exists in the MySQL database. The system shows "DATABASE OPERATION FAILED" status badge, "Unable to Complete Registration" heading, and the specific error message: "A student account with this email address is already registered in the MySQL database."



2.5 Output 5: Password Verification Error (Bcrypt password_verify)

Login page displaying "Incorrect password entered. Please try again." error message when password_verify() fails to match the entered password against the stored bcrypt hash. This confirms that the bcrypt hashing from registration is correctly verified during authentication. The login form accepts Email ID or Student Code with password input, Remember Me checkbox, and Forgot Password link, demonstrating the end-to-end password security flow from registration (password_hash) to authentication (password_verify).



3. Key Questions & Technical Analysis

Q1. Is password_hash() used before storing passwords?

Yes. The registration module employs PHP's native password_hash() function with the PASSWORD_BCRYPT algorithm to securely hash all passwords before database insertion:

• **Bcrypt Algorithm:** password_hash($password, PASSWORD_BCRYPT) applies the Blowfish cipher-based bcrypt algorithm with a cost factor of 10 (default), producing a 60-character hash string prefixed with $2y$10$. Each hash includes a unique 22-character random salt, ensuring identical passwords produce different hash outputs.

• **Storage Format:** The resulting hash is stored in the password column (VARCHAR(255)) of the students table. The column size accommodates future algorithm upgrades that may produce longer hashes.

• **No Plaintext Storage:** At no point is the raw password written to the database. The password variable is immediately hashed after server-side validation and the original value is not retained in memory beyond the scope of the registration handler.

• **Verification Compatibility:** The stored bcrypt hash is compatible with password_verify($input, $storedHash) used in the login module (Practical 10), enabling secure credential verification without reversing the hash.

Q2. Is duplicate email/username checked?

Yes. Duplicate detection is enforced at both the application layer and database constraint level:

• **Application-Level Check:** Before executing the INSERT query, the registration handler runs a SELECT COUNT(*) FROM students WHERE email = :email prepared statement. If the count is greater than zero, the system immediately halts registration and redirects to a descriptive error page.

• **Database UNIQUE Constraint:** The students table schema defines a UNIQUE constraint on the email column, providing a database-level safety net. Even if the application check is bypassed (e.g., concurrent requests), MySQL will reject the duplicate INSERT with a constraint violation error caught by the PDO exception handler.

• **Roll Number Uniqueness:** Similarly, the roll_number column carries a UNIQUE constraint to prevent duplicate student registrations under the same institutional roll number.

Q3. Are frontend and backend validations consistent?

Yes. The registration system implements a dual-layer validation strategy ensuring consistent enforcement on both client and server sides:

• **Frontend (JavaScript) Validation:** Real-time form validation checks required fields, email format (regex pattern), mobile number format (10-digit pattern), password strength (minimum 8 characters with complexity requirements for uppercase, lowercase, numbers, and symbols), password confirmation match, and Terms & Conditions acceptance.

• **Backend (PHP) Validation:** Server-side validation replicates all frontend checks using filter_input(INPUT_POST, ...), filter_var($email, FILTER_VALIDATE_EMAIL), strlen() for length constraints,

and preg_match() for pattern enforcement. This ensures security even if JavaScript is disabled or frontend validation is bypassed.

• **Sanitization Layer:** All user inputs are sanitized using htmlspecialchars() with ENT_QUOTES flag and trim() to prevent XSS attacks and strip leading/trailing whitespace before database insertion. The sanitized values are bound as parameters in PDO prepared statements for SQL injection prevention.

• **Error Message Consistency:** Both frontend and backend produce semantically identical error messages so users receive a consistent experience regardless of which validation layer triggers first.

4. GitHub Repository Link

GitHub Repository URL: https://github.com/smit-2406/studenthub-website

5. Conclusion & Learning Outcomes

Through the execution of Practical 9, secure user registration, bcrypt password hashing, duplicate email prevention, and PDO prepared statement-based database insertion were successfully implemented within the StudentHub Portal. The registration module demonstrates industry-standard security practices including never storing plaintext passwords, enforcing dual-layer (client + server) validation, sanitizing all user inputs against XSS, preventing SQL injection through parameterized queries, and providing descriptive error feedback for duplicate registrations and database operation failures. The practical reinforced critical web security concepts: the irreversible nature of bcrypt hashing with unique salts per record, the importance of UNIQUE constraints at the database level as a safety net beyond application logic, and the necessity of server-side validation as the authoritative security boundary. These foundations directly support the login/logout authentication system in Practical 10 using password_verify() against the stored bcrypt hashes. This practical fulfills Course Outcomes CO1, CO3, and CO5.